BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Lambda ve Built-in Fonksiyonlar

EBRU AYDOĞAN

Lambda (Anonim) Fonksiyonlar

- Normalde def ile fonksiyon tanımlarız. Ama bazen sadece bir kez kullanacağımız, ismi olmayan "tek kullanımlık" fonksiyonlara ihtiyaç duyarız.
- Lambda fonksiyon tanımlama:
  - lambda arguments : expression

BTK AKADEMİ

Lambda (Anonim) Fonksiyonlar

```python
# Klasik
def kare_al(x):
    return x**2

# Lambda
kare = lambda x: x**2
print(kare(5)) # 25
```

BTK AKADEMİ

Lambda (Anonim) Fonksiyonlar

- Birden fazla Parametre:
  - Örnek: İki sayıyı toplayan lambda:
    - topla = lambda a, b: a + b

BTK AKADEMİ

Lambda (Anonim) Fonksiyonlar

- İç İçe (Nested) Lambda
  - Bir lambda fonksiyonu, başka bir lambda fonksiyonunu döndürebilir (return edebilir).

```
# Bir fonksiyon yazalım, bu fonksiyon bize çarpma işlemi yapan
# başka fonksiyonlar üretsin.
katlayici = lambda n : lambda a : a * n

# Şimdi bu fabrikayı kullanarak iki farklı fonksiyon üretelim:
iki_katini_al = katlayici(2)
uc_katini_al = katlayici(3)

print(iki_katini_al(10)) # Sonuç: 20
print(uc_katini_al(10))  # Sonuç: 30
```

BTK AKADEMİ

Map Fonksiyonu (Dönüştürücü)

- **Senaryo:** Bir sayı listesindeki her elemanın karesini alıp yeni bir liste yapalım.
  - Klasik Yol (For Döngüsü):

```
sayilar = [1, 2, 3, 4]
kareler = []
for n in sayilar:
    kareler.append(n**2)
```

BTK AKADEMİ

Map Fonksiyonu (Dönüştürücü)

- Map+fonksiyon
  - Kuralı bir fonksiyona hapsediyoruz, map ile listeye fırlatıyoruz.

```
sayilar = [1, 2, 3, 4, 5]

def karesini_hesapla(x):
    return x ** 2

# map(kural_fonksiyonu, veri_listesi)
kareler = list(map(karesini_hesapla, sayilar))
```

BTK AKADEMİ

Map Fonksiyonu (Dönüştürücü)

- Map + Lambda
  - Fonksiyon tanımlama zahmetine girmeden, "tek kullanımlık" kuralı satır içine yazıyoruz.

```
sayilar = [1, 2, 3, 4, 5]

kareler = list(map(lambda x: x ** 2, sayilar))
```

BTK AKADEMİ

Filter Fonksiyonu

- **Senaryo:** Karışık bir sayı listesindeki sadece negatif (0'dan küçük) sayıları ayıklayıp yeni bir liste yapalım.
  - Klasik Yol

```
sayilar = [10, -5, 20, -3, 0, -12, 7]
negatifler = []

for x in sayilar:
    if x < 0:
        negatifler.append(x)

# Çıktı: [-5, -3, -12]
```

BTK AKADEMİ

Filter Fonksiyonu

- Filter+fonksiyon
  - Şartı bir fonksiyona devrediyoruz, filter ile listeyi süzgeçten geçiriyoruz.

```
sayilar = [10, -5, 20, -3, 0, -12, 7]

def negatif_mi(x):
    return x < 0  # True veya False döner

# filter(kural_fonksiyonu, veri_listesi)
negatifler = list(filter(negatif_mi, sayilar))
```

BTK AKADEMİ

Filter Fonksiyonu

- Filter + Lambda
  - Fonksiyon isimlendirme zahmetinden kurtulup, süzgeci doğrudan satır içine yazıyoruz.

```
sayilar = [10, -5, 20, -3, 0, -12, 7]

negatifler = list(filter(lambda x: x < 0, sayilar))
```

BTK AKADEMİ

Filter Fonksiyonu

- **Senaryo:** Bir isim listesinden sadece "A" harfi ile başlayanları seçelim.
  - Klasik yol:

```
isimler = ["Ali", "Ayşe", "Mehmet", "Buse", "Arda"]
a_ile_baslayanlar = []

for ad in isimler:
    if ad.startswith("A"):
        a_ile_baslayanlar.append(ad)
# Çıktı: ['Ali', 'Ayşe', 'Arda']
```

BTK AKADEMİ

Filter Fonksiyonu

- **Senaryo:** Bir isim listesinden sadece "A" harfi ile başlayanları seçelim.
  - Filter+lambda

```
isimler = ["Ali", "Ayşe", "Mehmet", "Buse", "Arda"]

a_ile_baslayanlar = list(filter(lambda ad: ad.startswith("A"), isimler))
```

BTK AKADEMİ

Map vs. Filter

- **Senaryo:** Hepsini büyük harf yapmak (Map)

```
# Herkes büyük harf olur (3 eleman girer, 3 eleman çıkar)
sonuc = list(map(lambda x: x.upper(), isimler))
# ['ALİ', 'AYŞE', 'MEHMET']
```

BTK AKADEMİ

Map vs. Filter

- **Senaryo:** Sadece "A" ile Başlayanları Büyük Harf Yap (Filter + Map)

```
# Önce A ile başlayanları seç (Filter)
secilenler = filter(lambda x: x.startswith("a"), isimler)

# Sonra seçilenleri büyük harf yap (Map)
sonuc = list(map(lambda x: x.upper(), secilenler))
# ['ALİ', 'AYŞE']
```

BTK AKADEMİ

Map vs. Filter

- Tek satırda:

```
isimler = ["ali", "ayşe", "mehmet", "arda"]

# Sağdan sola oku: Önce filtrele, sonra büyük harfe çevir.
n_sonuc = list(map(lambda x: x.upper(), filter(lambda x:
x.startswith("a"), isimler)))

print(n_sonuc) # ['ALİ', 'AYŞE', 'ARDA']
```

BTK AKADEMİ

Gerçek Dünya Senaryosu

- **Senaryo:** Bir sosyal medya uygulamasından gelen kullanıcı verilerimiz var. Gönderi (post) sayısı 2'den az olan "pasif" kullanıcıları tespit etmek istiyoruz.

Gerçek Dünya Senaryosu

- Veri Yapısı:

```
users = [
    {"username": "ebru_aydogan", "posts": ["Python 101", "İleri Seviye Python", "Data Science"]},
    {"username": "btk_akademi", "posts": ["Duyuru", "Yeni Kurs"]},
    {"username": "yazilim_ogrencisi", "posts": ["Merhaba Dünya"]},
    {"username": "python_coder", "posts": []}
]
```

BTK AKADEMİ

Gerçek Dünya Senaryosu

- Klasik yol:

```
pasif_kullanicilar = []
for user in users:
    if len(user["posts"]) < 2:
        pasif_kullanicilar.append(user["username"])
```

BTK AKADEMİ

Gerçek Dünya Senaryosu

- Filter + Lambda:

```
# Önce objeyi filtreleyelim
filtre_sonucu = filter(lambda user: len(user["posts"]) < 2, users)
 # Sadece kullanıcı adlarını alalım (Map ile birleştirelim)
pasif_isimler = list(map(lambda u: u["username"], filtre_sonucu)) print(pasif_isimler)
# Çıktı: ['yazilim_ogrencisi', 'python_coder']
```

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- All Fonksiyonu (Hepsi True mu?)
  - Verilen bir iterable (liste vb.) içindeki TÜM elemanlar şartı sağlıyorsa True, bir tanesi bile bozuyorsa False döner.

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- **Senaryo:** Bir sınıftaki tüm öğrencilerin sınavdan geçip geçmediğini (en az 50 alıp almadıklarını) kontrol edelim.
  - Klasik yol:

```
notlar = [60, 85, 90, 45, 70]
hepsi_gecti = True

for n in notlar:
    if n < 50:
        hepsi_gecti = False
        break # Bir kişi bile kaldıysa döngüden çık
```

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- **Senaryo:** Bir sınıftaki tüm öğrencilerin sınavdan geçip geçmediğini (en az 50 alıp almadıklarını) kontrol edelim.
  - All ile (İleri seviye):

```
notlar = [60, 85, 90, 45, 70]

# "Her n için, n >= 50 mi?" kontrolü
sonuc = all([n >= 50 for n in notlar])
# Çıktı: False (Çünkü 45 var)
```

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- Any Fonksiyonu (En Az Biri True mu?)
  - Iterable içindeki elemanlardan **EN AZ BİRİ** şartı sağlıyorsa True döner. Hepsi yanlışsa False döner.

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- **Senaryo:** "Sınıfta hiç 100 alan (dahi) öğrenci var mı?" kontrolü yapalım.
  - Klasik yol:

```
notlar = [60, 85, 90, 45, 70]
dahi_var_mi = False

for n in notlar:
    if n == 100:
        dahi_var_mi = True
        break
```

BTK AKADEMİ

Mantıksal Doğrulama (Any & All)

- **Senaryo:** "Sınıfta hiç 100 alan (dahi) öğrenci var mı?" kontrolü yapalım.
  - Any ile:

```
notlar = [60, 85, 90, 45, 70]

sonuc = any([n == 100 for n in notlar])
# Çıktı: False
```

BTK AKADEMİ

Sorted Fonksiyonu (Veri Sıralama)

- sorted() fonksiyonu, orijinal listeyi bozmadan bize sıralanmış yeni bir liste döndürür.

```
sayilar = [45, 12, 89, 3, 27]
sirali_sayilar = sorted(sayilar)
# Çıktı: [3, 12, 27, 45, 89]

# Tersten sıralama (Büyükten Küçüğe)
ters_sirali = sorted(sayilar, reverse=True)
# Çıktı: [89, 45, 27, 12, 3]
```

BTK AKADEMİ

Sorted Fonksiyonu (Veri Sıralama)

- Dictionary Üzerinde Basit Sıralama (Key)
  - **Senaryo:** Sözlük anahtarlarını veya değerlerini kendi içinde sıralamak.

```
stok = {"Elma": 50, "Muz": 20, "Çilek": 100}

# Sadece anahtarları (A-Z) sıralar
sirali_anahtar = sorted(stok)
# ['Çilek', 'Elma', 'Muz']
```

BTK AKADEMİ

Sorted Fonksiyonu (Veri Sıralama)

- Key + Lambda ile Profesyonel Sıralama
  - **Senaryo**: Sözlüklerden oluşan bir listeyi, içindeki belirli bir veriye göre (örneğin fiyat) sıralayalım..

```
urunler = [
    {"ad": "Laptop", "fiyat": 35000},
    {"ad": "Mouse", "fiyat": 750},
    {"ad": "Klavye", "fiyat": 1200}
]

# "Her bir x (sözlük) için git ve onun 'fiyat' değerine bak"
sirali_urunler = sorted(urunler, key=lambda x: x["fiyat"])
```

BTK AKADEMİ

Sorted Fonksiyonu (Veri Sıralama)

- **Soru:** İsim listesini alfabetik değil de, isimlerin **uzunluklarına** göre (en kısa isimden en uzuna) nasıl sıralarız?

```
isimler = ["Ebru", "Ali", "Abdullah", "Can"]
uzunluk_sirali = sorted(isimler, key=lambda x: len(x))
# Sonuç: ['Ali', 'Can', 'Ebru', 'Abdullah']
```

BTK AKADEMİ

Uygulama 1: Veri Temizleme

- Veri:
  - ham_veriler = ["Ahmet", " ", "Meryem", None, "Ali", " ", "Selin"]
- İstenen:
  - 1. None ve sadece boşluktan oluşan verileri **filter** ile ayıklayın.
  - 2. Kalan isimlerin sağındaki/solundaki boşlukları **map** ve **strip()** ile temizleyin.
  - 3. Sonucu temiz_isimler listesi olarak ekrana basın.

BTK AKADEMİ

Uygulama 2: Not Analiz Sistemi

- **Veri:** ogrenciler = [

  {"ad": "Lale", "notlar": [70, 80, 90]},

  {"ad": "Deniz", "notlar": [40, 50, 30]},

  {"ad": "Arda", "notlar": [90, 95, 100]},

  {"ad": "Zeynep", "notlar": [60, 60, 70]}]

- İstenen:
  - map ve lambda kullanarak her öğrenciye "ortalama" adında yeni bir anahtar ve hesapladığınız ortalamayı ekleyin.
  - filter kullanarak ortalaması 70 ve üzeri olan "başarılı" öğrencileri seçin.
  - sorted ve lambda kullanarak tüm listeyi ortalamaya göre büyükten küçüğe sıralayın.

BTK AKADEMİ